

SIMATS ENGINEERING

ASSESSMENT TOOL 2

COURSE CODE: CSA51

COURSE NAME: Cryptography and Network Security

COURSE OUTCOME COVERED

CO5: Develop the network security strategies based on the study of viruses, worms, firewall architectures, and IDS/IPS functionalities.CO (BL5 , BL6)

Assessment Tool Weightage: 20%

QUESTIONS: 5

TOTAL MARKS: 50

Annexure A

PSEUDOCODE WRITING TASK (ALGORITHM DESIGN)

Code of Conduct:

I Challa Priyadarshini(192311118) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: CH Priyadarshini

Annexure A

PSEUDOCODE WRITING TASK (ALGORITHM DESIGN)

1. Write pseudocode to simulate a simplified TLS handshake process, including client request, server authentication using a digital certificate, session key generation, and secure data exchange.
2. Develop pseudocode for a firewall rule engine that filters incoming packets based on IP address, port number, and protocol, and decides whether to allow or block traffic.
3. Write pseudocode to detect and classify malware behavior (virus, worm, or Trojan) based on propagation method, file modification patterns, and user interaction.
4. Design pseudocode for an Intrusion Detection System (IDS) that uses both signature-based and anomaly-based detection to identify suspicious network activity.
5. Create pseudocode for an Intrusion Prevention System (IPS) that monitors network traffic in real time and automatically blocks malicious requests while logging intrusion attempts.

1. Pseudocode for a Simplified TLS Handshake

Aim

To simulate a simplified TLS handshake in which a client requests a secure connection, the server authenticates itself using a digital certificate, both sides establish a session key, and encrypted data is exchanged securely.

Input

- Client identity and supported cryptographic parameters
- Server digital certificate
- Server private key
- Random values generated by client and server
- Application data to be transmitted

Output

A successfully established secure session and encrypted/decrypted application data.

Pseudocode

ALGORITHM Simplified_TLS_Handshake

BEGIN

Client -> Server: SEND ClientHello

 Include supported TLS versions

 Include supported cipher suites

 Generate ClientRandom

Server receives ClientHello

Server -> Client: SEND ServerHello

 Select TLS version

 Select cipher suite

 Generate ServerRandom

Server -> Client: SEND DigitalCertificate

Client receives DigitalCertificate

IF VerifyCertificate(DigitalCertificate) = FALSE THEN

 DISPLAY "Server authentication failed"

 TERMINATE connection

END IF

Client verifies that the certificate belongs to the server

Client verifies certificate validity and trust chain

Generate shared secret using secure key-establishment method

Client and Server derive SessionKey
SessionKey = KeyDerivation(ClientRandom,
ServerRandom,
SharedSecret)

Client -> Server: SEND HandshakeFinished
Authenticate previous handshake messages

Server -> Client: SEND HandshakeFinished
Authenticate previous handshake messages

IF HandshakeFinished messages are valid THEN
SET SecureConnection = TRUE
ELSE
TERMINATE connection
END IF

IF SecureConnection = TRUE THEN
Client encrypts ApplicationData using SessionKey
Client -> Server: SEND EncryptedData

Server decrypts EncryptedData using SessionKey

Server processes ApplicationData

Server encrypts Response using SessionKey
Server -> Client: SEND EncryptedResponse

Client decrypts EncryptedResponse
END IF
END

Explanation

The client begins by sending a ClientHello containing supported protocol parameters and a random value. The server selects compatible parameters and sends its certificate. The client validates the certificate before continuing. A secure key-establishment procedure is then used to derive shared session keying material. After the handshake is authenticated, application data is encrypted using the session key. The simplified algorithm intentionally omits many details of real TLS, such as extensions, certificate-chain processing, transcript hashes, and the exact TLS 1.3 message sequence.

2. Pseudocode for a Firewall Rule Engine

Aim

To design a rule engine that examines incoming packets using source IP address, destination IP address, port number, and protocol, and decides whether the packet should be allowed or blocked.

Input

- Incoming packet
- Source IP address
- Destination IP address
- Source/destination port
- Protocol such as TCP, UDP, or ICMP
- Ordered firewall rules

Output

ALLOW or BLOCK decision, together with a security log entry.

Pseudocode

ALGORITHM Firewall_Rule_Engine

INPUT:

Packet

RuleList

BEGIN

Extract SourceIP from Packet

Extract DestinationIP from Packet

Extract SourcePort from Packet

Extract DestinationPort from Packet

Extract Protocol from Packet

FOR each Rule in RuleList DO

IF Rule.Protocol = ANY OR Rule.Protocol = Protocol THEN

IF Rule.SourceIP = ANY OR Rule.SourceIP matches SourceIP THEN

IF Rule.DestinationIP = ANY OR

Rule.DestinationIP matches DestinationIP THEN

IF Rule.Port = ANY OR

Rule.Port matches DestinationPort THEN

IF Rule.Action = ALLOW THEN

LOG("Packet allowed", Packet, Rule.ID)

RETURN ALLOW

```

        ELSE IF Rule.Action = BLOCK THEN
            LOG("Packet blocked", Packet, Rule.ID)
            RETURN BLOCK
        END IF

    END IF
END IF
END IF
END IF

END FOR

LOG("No rule matched; applying default policy")

IF DefaultPolicy = ALLOW THEN
    RETURN ALLOW
ELSE
    RETURN BLOCK
END IF
END

```

Explanation

Firewall rules are checked in a defined order. Each rule compares relevant packet fields with the rule conditions. If a matching rule is found, its action is executed. A secure firewall normally uses a default-deny policy unless there is a justified reason to use a different default. Logging is important for troubleshooting, incident investigation, and auditing.

3. Pseudocode for Malware Behavior Detection and Classification

Aim

To detect suspicious malware behavior and classify it as virus, worm, or Trojan based on propagation method, file modification patterns, and user interaction.

Input

- Process and file activity
- File modification and replication events
- Network connection and scanning behavior
- User interaction information
- Persistence and execution indicators

Output

Malware classification: VIRUS, WORM, TROJAN, or UNKNOWN, plus a recommended response.

Pseudocode

ALGORITHM Detect_And_Classify_Malware

INPUT:

ProcessEvents
FileEvents
NetworkEvents
UserInteraction

BEGIN

virusScore <- 0
wormScore <- 0
trojanScore <- 0

IF ProcessEvents show modification or attachment
of malicious code to other executable files THEN
virusScore <- virusScore + 2
END IF

IF FileEvents show repeated self-replication
into other files or removable media THEN
virusScore <- virusScore + 2
END IF

IF NetworkEvents show automatic scanning of many hosts THEN
wormScore <- wormScore + 2
END IF

IF NetworkEvents show repeated attempts to exploit
vulnerable services without normal user action THEN
wormScore <- wormScore + 3
END IF

IF ProcessEvents show disguised or unauthorized software
establishing persistence THEN
trojanScore <- trojanScore + 2
END IF

IF UserInteraction = REQUIRED_FOR_INITIAL_EXECUTION THEN
trojanScore <- trojanScore + 2
END IF

IF NetworkEvents show command-and-control communication
after suspicious software execution THEN
trojanScore <- trojanScore + 2
END IF

```

IF wormScore >= virusScore AND wormScore >= trojanScore
  AND wormScore >= 3 THEN
  Classification <- "WORM"

ELSE IF virusScore >= wormScore AND virusScore >= trojanScore
  AND virusScore >= 3 THEN
  Classification <- "VIRUS"

ELSE IF trojanScore >= virusScore AND trojanScore >= wormScore
  AND trojanScore >= 3 THEN
  Classification <- "TROJAN"

ELSE
  Classification <- "UNKNOWN / NEEDS ANALYSIS"
END IF

LOG("Malware classification", Classification)

IF Classification = "WORM" THEN
  ISOLATE_HOST()
  BLOCK_SUSPICIOUS_LATERAL_TRAFFIC()

ELSE IF Classification = "VIRUS" THEN
  QUARANTINE_INFECTED_FILES()
  SCAN_RELATED_FILES()

ELSE IF Classification = "TROJAN" THEN
  TERMINATE_MALICIOUS_PROCESS()
  BLOCK_COMMAND_AND_CONTROL()
  QUARANTINE_SUSPICIOUS_FILE()
END IF

RETURN Classification
END

```

Explanation

The algorithm uses behavior rather than relying on a single filename or signature. A worm receives a high score when it automatically scans or attacks many systems. A virus receives a high score when it modifies or attaches itself to other files and replicates through them. A Trojan receives a high score when suspicious software is disguised as legitimate software, requires execution or user interaction, establishes persistence, or communicates with command-and-control infrastructure. Real malware can display characteristics of more than one category, so the classification should be treated as an analytical aid rather than an absolute label.

4. Pseudocode for a Hybrid Intrusion Detection System

Aim

To design an IDS that uses both signature-based and anomaly-based detection to identify suspicious network activity.

Input

- Live network packets or network-flow records
- Known attack signatures
- Normal traffic profile
- Thresholds and security policies

Output

Normal or suspicious classification, alert severity, evidence, and event log.

Pseudocode

ALGORITHM Hybrid_IDS

INPUT:

NetworkTraffic
SignatureDatabase
NormalProfile
Thresholds

BEGIN

WHILE NetworkTraffic is available DO

 Packet <- GET_NEXT_PACKET(NetworkTraffic)

 Features <- EXTRACT_FEATURES(Packet)

 SourceIP
 DestinationIP
 SourcePort
 DestinationPort
 Protocol
 PacketSize
 ConnectionRate
 PayloadIndicators

 signatureMatch <- MATCH_SIGNATURE(
 Features,
 SignatureDatabase)

 anomalyScore <- CALCULATE_ANOMALY_SCORE(
 Features,
 NormalProfile)

```

IF signatureMatch = TRUE THEN
    severity <- "HIGH"
    reason <- "Known attack signature detected"

    CREATE_ALERT(
        SourceIP,
        DestinationIP,
        severity,
        reason)

ELSE IF anomalyScore >= Thresholds.High THEN
    severity <- "HIGH"
    reason <- "Strong deviation from normal behavior"

    CREATE_ALERT(
        SourceIP,
        DestinationIP,
        severity,
        reason)

ELSE IF anomalyScore >= Thresholds.Medium THEN
    severity <- "MEDIUM"
    reason <- "Potentially suspicious anomaly"

    LOG_EVENT(
        SourceIP,
        DestinationIP,
        severity,
        reason)

ELSE
    LOG_EVENT("Normal traffic", SourceIP, DestinationIP)
END IF

END WHILE
END

```

Explanation

Signature-based detection is effective for known attacks because it compares traffic against known malicious patterns. Anomaly-based detection compares current behavior with a baseline of normal activity and can identify previously unseen or modified attacks. Combining both approaches provides broader coverage. Thresholds must be carefully tuned because an overly sensitive anomaly detector can generate excessive false positives.

5. Pseudocode for an Intrusion Prevention System

Aim

To design an IPS that monitors traffic in real time, detects malicious requests, automatically blocks high-confidence threats, and records intrusion attempts for security analysis.

Input

- Real-time network traffic
- Attack signatures and threat indicators
- Behavioral/anomaly rules
- Firewall/blocklist interface
- Logging system

Output

Allowed or blocked traffic and a detailed intrusion log.

Pseudocode

ALGORITHM Real_Time_IPS

INPUT:

NetworkTraffic
SignatureDatabase
ThreatIndicators
IPSRules

BEGIN

WHILE NetworkTraffic is available DO

 Request <- GET_NEXT_REQUEST(NetworkTraffic)

 Features <- INSPECT(Request)

 signatureMatch <- MATCH_SIGNATURE(
 Features,
 SignatureDatabase)

 threatMatch <- MATCH_THREAT_INDICATORS(
 Features,
 ThreatIndicators)

 anomalyScore <- CALCULATE_ANOMALY(
 Features,
 IPSRules)

 IF signatureMatch = TRUE THEN

```

    severity <- "HIGH"
    action <- "BLOCK"
    reason <- "Known malicious signature"

ELSE IF threatMatch = TRUE THEN

    severity <- "HIGH"
    action <- "BLOCK"
    reason <- "Known malicious indicator"

ELSE IF anomalyScore >= IPSRules.BlockThreshold THEN

    severity <- "HIGH"
    action <- "BLOCK"
    reason <- "High-confidence anomalous behavior"

ELSE IF anomalyScore >= IPSRules.AlertThreshold THEN

    severity <- "MEDIUM"
    action <- "ALERT"
    reason <- "Suspicious behavior"

ELSE

    severity <- "LOW"
    action <- "ALLOW"
    reason <- "No malicious evidence"

END IF

IF action = "BLOCK" THEN

    DROP(Request)

    ADD_TO_TEMPORARY_BLOCKLIST(
        Request.SourceIP,
        Request.DestinationIP)

    LOG_INTRUSION(
        Time,
        Request.SourceIP,
        Request.DestinationIP,
        severity,
        reason)

ELSE IF action = "ALERT" THEN

```

```

        FORWARD(Request)

    CREATE_ALERT(
        Time,
        Request.SourceIP,
        Request.DestinationIP,
        severity,
        reason)

ELSE

    FORWARD(Request)

END IF

END WHILE
END

```

Explanation

An IPS operates inline, meaning traffic can be stopped before reaching the protected service. The algorithm first extracts security-relevant features and checks them against known signatures and threat indicators. It also evaluates anomaly behavior. High-confidence malicious traffic is dropped and logged. Medium-confidence traffic may be allowed while generating an alert, depending on organizational policy. Temporary blocking helps contain repeated attacks while avoiding unnecessarily permanent blocks.

Important Design Considerations

- Use high-confidence rules for automatic blocking to reduce false positives.
- Maintain complete logs containing time, source, destination, reason, and action.
- Use rate limiting and temporary blocklists to contain repeated attacks.
- Monitor IPS performance so inspection does not become a network bottleneck.
- Review detection rules regularly and update signatures and threat intelligence.
- Provide a safe exception mechanism for legitimate traffic that is incorrectly blocked.

Conclusion

The five pseudocode algorithms demonstrate how security concepts can be converted into logical, implementable procedures. The simplified TLS handshake shows how authentication and session-key establishment lead to secure data exchange. The firewall algorithm demonstrates rule-based packet filtering. The malware classifier uses behavioral characteristics to distinguish virus, worm, and Trojan activity. The IDS combines known signatures with anomaly detection, while the IPS extends detection into real-time prevention by automatically blocking high-confidence attacks.

These algorithms are intentionally simplified for academic algorithm-design purposes. A production implementation must use tested cryptographic libraries, standards-compliant protocol implementations, secure configuration, careful error handling, performance testing, and comprehensive monitoring.

